

FINM 32000 Practice Midterm Exam

April 2026

Closed book, closed notes, closed devices. No collaboration.

Problems (and problem parts) may have different weights.

Wherever possible, your answers should be explicit. Basic operations on explicit numbers may be left unsimplified, for example $\binom{9}{8} + (7 - \log(6 \times 5/4))e^3 - 2.1$ is an acceptable format.

Final answers involving integrals, or the normal cdf (N or Φ), or the Black-Scholes formula will not get full credit, unless it is impossible to evaluate more explicitly.

If taking finite differences, two-sided (central) differences, if available, are preferable to one-sided differences, unless otherwise directed.

Assume frictionless markets and absence of arbitrage, unless otherwise directed.

Problem 1

The following table has time-0 price data for European-style options on RUT, the Russell 2000 index, with expiry $T = 25/365$. The (continuously compounded) interest rate is $r = 0.0432$. Let S_T denote the time- T level of RUT.

Strike	Call Price	Put Price
2280	44.7	59.2
2290	39.9	
2300	35.4	

- (a) Using the call and put prices at strike 2280, find the time-0 forward price F_0 of RUT, for delivery at the same time T as the options expiry.
- (b) Find the time-0 value of a forward contract on RUT with delivery price 2200 and delivery date T .
- (c) Approximate the risk-neutral probability that $2287 < S_T < 2293$.
- (d) Approximate the risk-neutral probability that $S_T > 2290$.
- (e) Fill in the blank price for the 2300-strike put.

Problem 2

A non-dividend-paying stock S has dynamics $S_0 = 10$, and for $t = 1, 2, 3$

$$S_t - S_{t-1} = \begin{cases} +2 & \text{with probability 50\%} \\ -2 & \text{with probability 50\%} \end{cases}$$

under risk-neutral measure. The interest rate is zero.

- (a) Find the time-0 price of an up-and-out put on S , with knockout barrier 11, strike 9, and expiry 3. The barrier is monitored at all times $t \in \{1, 2, 3\}$.
- (b) Find the time-0 price of a *one-touch* option on S with barrier 11 and expiry 3. This contract pays 1 dollar at the first time $t \in \{1, 2, 3\}$ such that $S_t \geq 11$. If $\max(S_1, S_2, S_3) < 11$ then the one-touch contract pays zero.

Unlike parts (a,b), consider in part (c) continuous-time dynamics for a non-dividend-paying stock X that follows Geometric Brownian motion with volatility 20%, starting at $X_0 = 10$. Interest rates are still zero.

- (c) Find, with justification, the time-0 price of an up-and-out put on X with barrier 12, strike 12, and expiry 9. The barrier is monitored at all times in the continuous-time interval $0 \leq t \leq 9$.

Problem 3

A fair die, by definition, generates independent rolls that are uniformly distributed on $\{1, 2, 3, 4, 5, 6\}$.

- (a) A fair die is rolled 3 times.

You receive the dollar amount shown on just *one* of those three rolls – the first or second or third roll – it's your choice, but if you want the n th roll, you must *exercise* that choice immediately after seeing the n th roll, before seeing the $(n + 1)$ th roll.

Find your expected payoff, if you exercise according to the stopping time that maximizes your expected payoff.

- (b) Suppose you play the unique strategy that attains this maximal expected payoff. Find the probability that your exercise time is time 3 (the time immediately after the third roll is revealed). In other words, this is the probability that you wait until the very end to exercise.

Problem 4

You are pricing a contract that pays, at expiry $T = 2.0$, some function $f(S_T)$ where S is a non-dividend-paying stock that follows Geometric Brownian motion with volatility $\sigma = 0.6$.

Let $C(S, t)$ be the contract's time- t price given time- t stock price S . Then C satisfies the PDE

$$\frac{\partial C}{\partial t} + \frac{1}{2}S^2\sigma^2\frac{\partial^2 C}{\partial S^2} = 0 \quad (1)$$

if the interest rate $r = 0$.

(a) For general r including $r \neq 0$, what would be the PDE?

In parts (b) and (c) assume $r = 0$, so use the PDE (1), rather than using your part (a) answer.

Working from right to left in the grid with spacings $\Delta t = 0.1$ and $\Delta S = 2$, suppose that you've used Crank-Nicolson as we defined in class, solving a linear system to compute finite difference approximations $\mathbb{C}(S, 1.2)$ for all S , and then solving another linear system to compute finite difference approximations $\mathbb{C}(S, 1.1)$ for all S .

(Notation: C is exact solution, $\mathbb{C}$ is the Crank-Nicolson approximate solution.)

S=152	34.84	32.95
S=150		31.73
S=148	32.40	30.53
	t=1.1	t=1.2

(b) You issue a “print” command, to see the solved grid, but a malevolent AI hides the $\mathbb{C}(150, 1.1)$ output. Reconstruct the hidden $\mathbb{C}(150, 1.1)$ output.

(c) The *charm* or *delta decay* of this contract is defined to be

$$\frac{\partial^2 C}{\partial t \partial S}$$

Based on the grid output, approximate the time-1.1 charm given $S_{1.1} = 150$.

Problem 5

The following Python notebook tries to calculate the implied volatility of a European put option on an underlying whose forward price is given. Find and correct the 3 errors in the code. Do not propose more than 3 corrections.

```

from copy import copy
import numpy as np
from scipy.optimize import bisect, brentq
from scipy.stats import norm

```

```

class CallOption:

```

```

    def __init__(self, K, T, price=None):
        self.K, self.T, self.price = K, T, price

```

```

class GBMdynamics:

```

```

    def __init__(self, X, r, rGrow, sigma=None):
        self.X, self.r, self.rGrow, self.sigma = X, r, rGrow, sigma

    def update_sigma(self, sigma):
        self.sigma = sigma
        return self

```

```

class AnalyticEngine:

```

```

    def __init__(self):
        pass

    def BSpriceCall(self, dynamics, contract):
        # Ignores contract.price if given, because this function calculates time-0 price based on the dynamics.

        F = dynamics.X*np.exp(dynamics.rGrow*contract.T)
        std = dynamics.sigma*np.sqrt(contract.T)
        d1 = np.log(F/contract.K)/std+std/2
        d2 = d1-std
        return np.exp(-dynamics.r*contract.T)*(F*norm.cdf(d1)-contract.K*norm.cdf(d2))

    def BSvega(self, dynamics, contract):
        # Returns time-0$ vega

        F = dynamics.X*np.exp(dynamics.rGrow*contract.T)
        std = dynamics.sigma*np.sqrt(contract.T)
        d1 = np.log(F/contract.K)/std+std/2
        return np.exp(-dynamics.r*contract.T)*F*norm.pdf(d1)*np.sqrt(contract.T)

    def IVofCall(self, dynamics, contract):
        # ignores dynamics.sigma, because this function solves for sigma. Returns time-0$ implied volatility

        if contract.price is None:
            raise ValueError('Contract price must be given')

        df = np.exp(-dynamics.r*contract.T) #discount factor
        F = dynamics.X*np.exp(dynamics.rGrow*contract.T)
        lowerbound = np.max([0, df*(F - contract.K)])
        C = contract.price
        if C < lowerbound:
            return np.nan
        if C == lowerbound:
            return 0
        if C >= F*df:
            return np.nan

        dynamics_try = copy(dynamics) # We "try" values of sigma until we find sigma that generates price C

        sigma_try = 0.2
        while self.BSpriceCall(dynamics_try.update_sigma(sigma_try), contract) > C:
            sigma_try /= 2
        while self.BSpriceCall(dynamics_try.update_sigma(sigma_try), contract) < C:
            sigma_try *= 2
        hi = sigma_try
        lo = hi/2

        impliedVolatility = bisect(lambda sigma: self.BSpriceCall(dynamics_try.update_sigma(sigma), contract), lo, hi)
        return impliedVolatility

```

```
# The values in this cell are correctly given:
K = 95          # strike of put option
T = 25/365      # expiry of put option
r = 0.0432      # interest rate
F = 102         # time-0 forward price for time-T delivery of underlying
putPrice = 3.5  # time-0 price of put option
```

```
impliedCallPrice = putPrice + np.exp(-r*T)*(F-K)
impliedCallK = F*F/K
```

```
engine = AnalyticEngine()
dynamics = GBMdynamics(X=F, r=r, rGrow=r, sigma=None)
```

```
impliedVolOfPut = engine.IVofCall(dynamics, CallOption(K=impliedCallK, T=T, price=impliedCallPrice))
```